TokenBrief — Autonomous AI Document Intelligence

Full Technical Architecture, Production Workflow & Comprehensive Viva Examination Guide

Project Type: Enterprise Document RAG & Workspace	**Tech Stack:** Next.js 14, FastAPI, Neon pgvector, Gemini 2.5
Author / Candidate: Tushar Kumar (ADGITM)	**Evaluation Mode:** Academic Viva & Production Deployment

1. Executive Summary & Problem Statement

What is TokenBrief? TokenBrief is an enterprise-grade AI Document Intelligence workspace. It allows users to upload complex, multi-page PDFs (such as academic syllabi, scientific research papers, data mining lab manuals, and professional resumes) and interact with them in real time.

The Problem with Traditional LLMs & Simple RAG:

- **The Context Limit & Lost-in-the-Middle Problem:** Uploading a 50-page PDF directly into a standard chat context either exceeds token limits or causes the AI to miss vital facts buried in the middle.
- **The Top-K Retrieval Trap:** Standard RAG only fetches the top 3-5 isolated text chunks (~2% of the document). Asking 'Give me an executive summary of this whole report' fails completely because 98% of the document is ignored.
- **Hallucinations:** Generic LLMs generate plausible-sounding but false answers when the document does not contain the answer.
- **Static, Boring Interaction:** Most document tools only offer a single chat box. They lack active learning tools like interactive test generation, multi-file comparisons, or visual knowledge graphs.

How TokenBrief Solves This: TokenBrief implements a **Two-Tier Hierarchical RAG Architecture** (Map-Reduce for full-document digests + Reciprocal Rank Fusion hybrid search for precision lookups), an **Abstention Guardrail Gate** that blocks hallucinations if confidence is below 25%, an **Interactive Quiz Engine** with domain-aware distractor synthesis, a **Multi-Document Matrix Comparator**, and an **Entity Knowledge Graph Canvas**.

2. Full Multi-Tier System Architecture

TokenBrief is decoupled into three independent, horizontally scalable tiers:

Tier	Technologies	Core Responsibilities
Frontend Client	Next.js 14 App Router, React 18, TypeScript, Tailwind CSS, Lucide Icons	Retentrix-style workspace layout, dynamic prompt suggestions, interactive quiz sliders, audio synthesis (Web Speech API), PDF export viewer, SVG knowledge graph.
Backend API	FastAPI (Python 3.11), Uvicorn, SQLAlchemy AsyncIO, PyPDF, ReportLab	RESTful endpoints, asynchronous chunking, Query Intent Router, Reciprocal Rank Fusion, Map-Reduce batching, JWT cookie auth, disposable email filtering.

AI / ML Engine	Gemini 2.5 Flash/Pro, sentence-transformers all-MiniLM-L6-v2, PyTorch CPU	Generates 384-dimensional dense embeddings, multi-turn grounded responses, structured JSON quiz parsing, and offline conceptual fallback synthesis.
Data & Vector Tier	PostgreSQL 16, pgvector extension, Neon Serverless Cloud (SSL)	Relational storage (users, documents, quizzes, annotations, telemetry logs) and HNSW cosine similarity index for sub-millisecond semantic search.

3. Step-by-Step Data Lifecycle & Execution Pipeline

Step 1: Document Upload & Structural Parsing

When a user uploads a PDF, the backend uses PyPDF to extract raw page text while preserving page numbers and section headers. A structure-aware chunker divides text into 400–600 token windows with a 50-token overlapping sliding window to prevent cutting sentences across boundaries.

Step 2: Vector Embedding & Storage

Each chunk is passed to the local `all-MiniLM-L6-v2` neural model to generate a 384-dimensional dense semantic vector. Chunks and embeddings are inserted into Neon PostgreSQL via `pgvector` with an HNSW cosine index for sub-millisecond similarity lookups.

Step 3: Query Intent Classification

Incoming user queries are analyzed before querying the database:
• **GLOBAL_SUMMARY:** Routes to pre-computed hierarchical Map-Reduce batches for 100% document coverage.
• **MULTI_DOC_COMPARE:** Gathers criteria across active documents into a comparative JSON matrix.
• **SPECIFIC_FACT:** Triggers Hybrid Search combining BM25 keyword frequency and vector cosine similarity via Reciprocal Rank Fusion (RRF).

Step 4: Hallucination Guardrail & Grounded Synthesis

Retrieved chunks are evaluated by an Abstention Gate. If similarity scores fall below 25%, the model halts and informs the user rather than hallucinating. Verified context is supplied to Google Gemini 2.5 Flash with strict grounding instructions and exact page citation enforcement.

4. Core Workspace Modules & Algorithms

• **AI Quiz Studio & Spaced Repetition:** Generates grounded MCQs with interactive sliders (3-15 questions) and difficulty levels. Employs a domain-aware question synthesizer that strips unformatted table dumps and broken sentence fragments. Distractors are drawn from the same subject context, option positions are uniformly randomized (A/B/C/D), and incorrect answers are tracked in `quiz_answers` for Spaced Repetition Review.

• **Side-by-Side Multi-Document Comparison:** Enables comparative analysis across 2+ files simultaneously. Generates a structured criteria-by-criterion matrix, shared topic pills, unique distinct attributes per document, and an executive verdict.

• **Interactive Entity Knowledge Graph:** Extracts core concepts, skills, tools, organizations, and methodologies from indexed documents and renders an interactive SVG network canvas with zoom, pan, and node inspection cards.

- **Real Analytics & Telemetry:** Logs query latency in milliseconds, token counts, model usage breakdown (Pro vs. Flash), and semantic cache hit ratios, visualized through zero-dependency responsive SVG charts.
- **Branded PDF Report Generation:** Compiles active chat briefings, citations, takeaways, and metrics into a multi-page, publication-grade downloadable PDF via ReportLab.

5. Top 25 Viva Examination Questions & Model Answers

The following questions represent the exact technical inquiries posed by university professors, project examiners, and technical evaluators.

Q1. What is TokenBrief and what primary real-world problem does it solve?

TokenBrief is an autonomous enterprise document intelligence platform built on Retrieval-Augmented Generation (RAG). It solves the limitations of traditional LLMs: context window overflow on long PDFs, the 'Top-K retrieval trap' where summaries ignore 98% of the document, hallucinations on unindexed queries, and lack of interactive learning tools.

Q2. Why did you choose RAG (Retrieval-Augmented Generation) instead of Fine-Tuning an LLM?

Fine-tuning is expensive, slow to retrain, alters model weights without guaranteeing factual recall, and cannot restrict access per user document. RAG separates knowledge storage from reasoning: private documents are indexed in a vector database, enabling instant document updates, zero training cost, verifiable page citations, and zero risk of proprietary data baking into static model weights.

Q3. Why did you select FastAPI for the backend over Django or Flask?

FastAPI is natively asynchronous (built on ASGI and Starlette), delivering up to 3x higher throughput than Flask or Django for I/O-bound tasks like vector searches, database queries, and external LLM streaming. It also provides automatic OpenAPI/Swagger documentation, strict Pydantic data validation, and native Python type hinting.

Q4. What is pgvector and how does vector similarity search work in your database?

pgvector is an open-source PostgreSQL extension that adds native support for high-dimensional vector embeddings. It allows storing 384-dimensional dense vectors directly alongside relational user data and querying them using cosine similarity operator (<=>) or inner product, eliminating the need for a separate third-party vector database.

Q5. Which vector indexing algorithm does your PostgreSQL database use, and why (HNSW vs IVFFlat)?

We use HNSW (Hierarchical Navigable Small World). While IVFFlat partitions vector space into clusters and requires frequent retraining, HNSW builds a multi-layer graph of vectors that allows logarithmic O(log N) search times, higher recall accuracy (>95%), and seamless incremental inserts without retraining the index.

Q6. What embedding model is used and what is its dimensional footprint?

We utilize sentence-transformers/all-MiniLM-L6-v2. It maps text sentences into a 384-dimensional dense vector space. It was chosen because it achieves state-of-the-art semantic search accuracy while executing in ~15ms on standard CPU hardware without requiring dedicated GPU acceleration.

Q7. What is Reciprocal Rank Fusion (RRF) and why do you use Hybrid Search?

Pure vector search understands semantics but can overlook specific alphanumeric codes, roll numbers, or exact product names. Pure keyword search (BM25) finds exact words but fails on synonyms. Hybrid Search runs both simultaneously and combines their rankings using Reciprocal Rank Fusion: RRF_Score = sum(1 / (60 + rank)). This guarantees both high semantic understanding and exact keyword precision.

Q8. How does your system handle long 50+ page PDFs without running out of context tokens?

We implement a Two-Tier Hierarchical Map-Reduce architecture. At upload time, the document is divided into 5-page batches. An asynchronous Map-Reduce pipeline digests each batch into structured bullet points stored in the document_sections table. When a user requests a global summary, the system synthesizes these pre-computed summaries in under 3 seconds with 100% document coverage.

Q9. How does your system prevent LLM hallucinations?

Through a dual-guardrail mechanism: 1) Pre-LLM Grounding Filter / Abstention Gate: If the cosine similarity score of the top retrieved chunk is below 25%, the system immediately halts and informs the user that the document lacks the necessary information. 2) Post-LLM Citation Verifier: Validates that every citation corresponds to an actual chunk boundary.

Q10. How does the AI Quiz Studio generate multiple-choice questions without option bias?

The quiz engine extracts verified facts, chapters, and concepts directly from document chunks. Distractors are drawn from the same subject context rather than generic templates. It utilizes an MCQ builder that shuffles the correct option among A, B, C, and D using random.shuffle, ensuring an even 25% distribution across all four choices.

Q11. What is the Spaced Repetition Review feature in the Quiz Studio?

When users submit quiz attempts, their selections and mistakes are saved in the quiz_answers relational table. The Spaced Repetition endpoint queries previously failed questions for that document, allowing users to re-test on their specific weak areas until mastery is achieved.

Q12. How is user authentication implemented and secured?

Authentication utilizes JSON Web Tokens (JWT) with HS256 encryption. Passwords are salted and hashed using bcrypt. Access tokens are stored in HttpOnly, SameSite=None, Secure cookies to protect against Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF).

Q13. Why use HttpOnly cookies for JWTs instead of localStorage in the frontend?

Storing JWTs in localStorage exposes tokens to any malicious JavaScript running on the page (XSS attacks). HttpOnly cookies cannot be read or modified by client-side JavaScript, ensuring that even if an XSS vulnerability exists, the attacker cannot steal the authentication session.

Q14. How do you handle Cross-Origin Resource Sharing (CORS) between Vercel and your backend?

In main.py, FastAPI's CORSMiddleware is configured with allow_credentials=True, allow_origin_regex covering https://.*\.vercel\.app, and explicit allowed headers and methods. In production, cookies specify SameSite='none' and Secure=True so browsers include credentials across different domains.

Q15. What is Semantic Caching and what performance benefits does it provide?

Semantic caching stores previously answered user queries alongside their embedding vectors in the query_semantic_cache table. When a new query arrives with a cosine similarity > 92% to a cached question, the cached answer is returned immediately (in <10ms), bypassing both the vector search and external LLM API calls.

Q16. How does the Entity Knowledge Graph visualizer render nodes without heavy external libraries?

The backend extracts entities (concepts, tools, frameworks, organizations) and inter-entity relationships during document digestion. The frontend uses a custom SVG canvas with a spring-embedded force simulation to compute 2D coordinate projections with interactive zooming, panning, and node inspection cards.

Q17. What happens if the Gemini API key is invalid, rate-limited, or network fails?

The system includes a resilient local neural synthesis fallback. When the Gemini API call encounters an authentication error (401) or timeout, the engine immediately diverts to the local synthesizer, generating clean conceptual summaries and grounded MCQs without throwing 500 errors to the client.

Q18. What is the role of the Query Intent Router?

Rather than treating every user input identically, the Query Intent Router uses rule-based and semantic classification to categorize queries into GLOBAL_SUMMARY, MULTI_DOC_COMPARE, or SPECIFIC_FACT. This ensures each query type triggers the most efficient and accurate retrieval algorithm.

Q19. Why Next.js 14 App Router for the frontend?

Next.js 14 App Router offers React Server Components (RSC) which minimize JavaScript bundle sizes sent to the client, automatic code splitting, optimized static generation, and edge middleware for route protection before private dashboard pages render.

Q20. What is the difference between Server Components and Client Components in your frontend?

Server Components render entirely on the server and transmit zero client-side JavaScript (used for layout scaffolding and initial HTML). Client Components ('use client') execute in the browser to provide interactivity, such as audio synthesis, sliders, sticky note annotations, and SVG canvas interactions.

Q21. How does the PDF Export briefing feature work under the hood?

The /api/export/pdf endpoint accepts the active conversation ID, fetches Q&A; history and source citations, and uses Python ReportLab to dynamically construct a formatted, multi-page PDF document containing branded headers, structured tables, and page numbering, streamed as a binary response.

Q22. Why did Vercel fail to deploy the Python backend directly, and how was it solved?

Vercel Serverless Functions have a maximum unzipped bundle limit of 500MB. PyTorch with CUDA dependencies exceeds 5.5GB. We solved this by decoupling the architecture: deploying the Next.js frontend to Vercel, and containerizing the FastAPI backend via a Dockerfile using CPU-only PyTorch (180MB) on Render/Railway.

Q23. How do you monitor and evaluate real-time system performance?

Every query executes within a telemetry wrapper that records the query text, response latency in milliseconds, token estimates, and model variant in the query_logs table. The Real Analytics dashboard presents rolling aggregations, model distribution donuts, and latency trends.

Q24. How does your database handle concurrent requests from multiple users?

Neon PostgreSQL utilizes connection pooling (PgBouncer) and SQLAlchemy's async_sessionmaker with AsyncEngine. All database operations are non-blocking coroutines running on the Python asyncio event loop, allowing thousands of concurrent read queries without blocking worker threads.

Q25. If given 6 more months, what three major enhancements would you implement?

1) Multimodal Table & Chart Parsing using ColPali / vision embeddings to query charts and diagrams directly. 2) Agentic Multi-Hop Reasoning using ReAct loops to solve complex cross-document queries requiring multiple steps. 3) Offline Local LLM Support using quantized GGUF models via llama.cpp for completely air-gapped enterprise deployments.